אלגוריתמים 2: הרצאות 4 – 3

26 באוקטובר 2026

גיא יער־און

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

1 Approximation Algorithms־ אלגוריתמי קירוב

1.1 חישוביות וסיבוכיות

הגדרה 1. תהי L שפה. $L \in P$ אם ורק אם קיימת מכונת טיורינג דטרמיניסטית M כך שמתקיים:
- לכל $x \in L$: $M(x) = 1$.
- לכל $x \notin L$: $M(x) = 0$.

הגדרה 2. תהי L שפה. $L \in NP$ אם ורק אם קיימת מכונת טיורינג לא דטרמיניסטית M כך שמתקיים:
- אם $x \in L$, קיימת ריצה כך ש$M(x) = 1$.
- אם $x \notin L$, לכל ריצה $M(x) = 0$.

משפט 3. הבעיה *Integer programming* מוגדרת להיות: בהינתן מטריצה A, האם קיים וקטור $x \in \mathbb{N}^n_{\geq 0}$ כך ש$Ax \geq b$. אזי, $IP \in NP$.

הגדרה 4. L שפה נקראת NP־קשה, אם לכל שפה $L' \in NP$ קיימת רדוקציית קארפ בזמן פולינומיאלי לL.

הגדרה 5. בעיית אופטימיזציה מוגדרת באמצעות רביעייה: $O = (I, Sol, m, type)$ באשר:
I היא קבוצת המופעים.
$Sol(i)$ - קבוצת הפתרונות האפשריים עבור המופע הi.
$m(i, s)$ היא פונקציית המדידה של הפתרון, עבור $i \in I, s \in Sol(i)$.
$type \in \{\min, \max\}$.
הערך האופטימלי מוגדר להיות:

$$opt(i) = \text{Type}_{s \in Sol(i)} m(i, s)$$

דוגמה 6. לדוגמה, נניח ונרצה למצוא את המסלול הקצר ביותר מs לt בגרף G. אזי זו בעיה מטיפוס מינימום וכן:

$$I = \{((G = (V, E), s, t) | s, t \in V\}$$

$$Sol((G=(V,E),s,t) = \{P|\text{P is an s-t path in G}\}$$

$$m((G=(V,E),s,t),P) = |P|$$

הגדרה 7. Decision version (גרסת ההכרעה) של בעיית אופטימיזציה: בהינתן פרמטר $k \in \mathbb{R}$
- אם $type = \max$ צריך להכריע האם קיים פתרון אפשרי $s \in Sol(i)$ כך שמתקיים $m(i,s) \geq k$
- אם $type = \min$ צריך להכריע האם קיים פתרון אפשרי $s \in Sol(i)$ כך שמתקיים $m(i,s) \leq k$

הגדרה 8. NPO היא מחלקת בעיות האופטימיזציה שגרסאות ההכרעה שלהן שייכות למחלקה NP. כלומר, בהינתן בעיית אופטימיזציה $O = (I, Sol, m, type)$ אזי בהינתן מופע i ופתרון מועמד s אנו יכולים:
- לקבוע האם $s \in Sol(i)$.
- לחשב ולהעריך את $m(i,s)$ - מדד ועלות הפתרון.

דוגמה 9. לדוגמה, בעיית הסוכן הנוסע (TSP) שמוגדרת להיות מציאת המסלול הקצר ביותר שמבקר בכל הקודקודים, היא בעיית אופטימיזציה שגרסת ההכרעה שלה הרי שייכת לNP, ולכן היא בNPO.

הגדרה 10. שפה או בעיית אופטימיזציה $O \in NPO$ נמצאת במחלקה PO אם קיים אלגוריתם שמוצא בזמן פולינומיאלי את הערך האופטימלי:

$$\text{opt}(i) = type_{s \in Sol(i)} m(i,s)$$

הגדרה 11. תהי $O \in NPO$ בעיית אופטימיזציה. אזי, הבעיה היא NP־שלמה אם גרסת גרסת ההכרעה המתאימה שלה היא NP־שלמה.

1.2 אלגוריתם קירוב של VC

נתבונן באלגוריתם פשוט וטריוואלי שמצליח לקרב בצורה די טובה את VC, למעשה: עוברים על כל הקשתות לפי סדר כלשהו. אם הקשת מכוסה כבר ע"י אחד הקודקודים - מתקדמים לאיטרציה הבאה, ואחרת מוסיפים את זוג הקודקודים שלה לכיסוי. נתבונן בתיאור האלגוריתם:

Algorithm: 2-Approximation for Vertex Cover

1. $A \leftarrow \emptyset$
2. While there is an uncovered edge $\{u,v\}$: add both u,v to A
3. **return** A

משפט 12. האלגוריתם הנ"ל הוא $approximation - 2$. כלומר, A היא כיסוי קודקודים ומתקיים $|A| \leq 2 \cdot |Opt|$

הוכחה: יהי M קבוצת הקשתות שנבחרו, קבוצה זו מהווה שידוך - כלומר: אין שתי קשתות בM שחולקות קודקוד משותף. נניח בשלילה שקיימות זוג קשתות $e_1 = (x,y), e_2 = (y,z)$ החולקות קודקוד משותף. בה"כ e_1 נבחרה ראשונה לכיסוי. אזי, בעת בחירת הקשת e_2 היא נבחרה כיוון שהיא לא הייתה מכוסה - כלומר y,z לא היו בכיסוי בעת כניסתה. אבל, $e_1 = (x,y)$ וקודקודיה כבר נכנסו לכיסוי. סתירה.

מכאן, הפתרון האופטימלי חייב להכיל לפחות קודקוד אחד מכל קשת שנמצאת בM, אחרת הוא לא כיסוי קודקודים. לכן מתקיים

$$|A| = 2 \cdot |M| \leq 2 \cdot |Opt|$$

שכן, A הוא מס' הקודקודים. וכן, הוא מהווה שיזוך שגודלו $|M|$ ־ שיוך נמדד בגודלו בקשתות, בכל צעד של בחירת קשת מוסיפים 2 קודקודים זרים לA, ואין בM קשתות חופפות, בהכרח A הוא מגודל פעמיים כפול מס' הקשתות בM. כנדרש.

■

הערה 13. הניתוח הדוק. כלומר, ישנן דוגמאות של גרפים בהם האלגוריתם יניב כיסוי בגודל בדיוק פעמיים מגודל הכיסוי האופטימלי.

הגדרה 14. APX היא משפחת בעיות הקירוב $O = (I, Sol, m, type)$ מתוך מחלקת NPO כך שעבור קבוע אבסולוטי כלשהו $c \geq 0$ קיים אלגוריתם בזמן פולינומי כך שלכל מופע $i \in I$ ניתן למצוא פתרון $s' \in Sol$ כך שמתקיים:
- אם $type = \min$, אזי $m(i, s') \leq c \cdot opt(i)$ עבור $c \geq 1$.
- אם $type = \max$ אזי $m(i, s') \geq c \cdot opt(i)$ עבור $c \leq 1$.

מסקנה 15. $VC \in APX$ עם $c = 2$.

נשאלת השאלה ההגיונית, האם אפשר יותר טוב מזה? ובכן, התשובה המפתיעה היא שלא יותר מדי.

משפט 16. $(Dinur,\ Safra\ 2005)$. תחת ההנחה כי $P \neq NP$, לא ניתן לקרב את בעיית כיסוי קודקודים ביחס טוב יותר מ1.3606.

משפט 17. $(Khot,\ Regev\ 2008)$. תחת הנחת משחקים ייחודיים, לא ניתן לקרב את בעיית כיסוי קודקודים ביחס טוב יותר מ2.

1.3 Max Cut

הגדרה 18. בהינתן גרף לא מכוון $G = (V, E)$, בעיית חתך מקסימלי מוגדרת להיות מציאת קבוצת קודקודים $S \subseteq V$ אשר ממקסמת את מס' הקשתות שנמצאות בחתך. כלומר, קשתות שמחברות בין קודקוד שנמצא בS לבין קודקוד שנמצא ב$V \backslash S$:

$$E(S, V \backslash S) = \{\{u, v\} \in E | u \in S \text{ and } v \in V \backslash S\}$$

בעיית Max Cut היא NP־קשה.

1.3.1 אלגוריתם הסתברותי

כיוון שמדובר בבעיה קשה, אין לנו דרך לפתור אותה בזמן פולינומי. במקום זאת, נסתכל על אלגוריתם הסתברותי שפותר אותה בהסתברות די טובה:

Algorithm: Random Cut (Probabilistic Approximation for Max-Cut)

1. Add each vertex $v \in V$ to S independently with probability $\frac{1}{2}$

נרצה לנתח את תוחלת מספר הקשתות שבחתך. תהי קשת כלשהי $\{u,v\}$. על מנת שהקשת תשתייך לחתך, בדיוק אחד מהקודקודים שלה צריך להיות בתוך הקבוצה S והשני צריך להיות מחוץ לה - $V\backslash S$. לכן:

$$\Pr[\{u,v\} \in E[S, V\backslash S] = \frac{1}{2}\cdot\frac{1}{2} + \frac{1}{2}\cdot\frac{1}{2} = \frac{1}{2}$$

שכן, ההסתברות שווה להסתברות בה"כ כי u בS וv לא בS, ועוד המקרה ההפוך. מלינאריות התוחלת, נקבל כי תוחלת גודל החתך:

$$\mathbb{E}[E(S, V\backslash S)] = \sum_{e\in E}\Pr[e \in E(S, V\backslash S)] = \sum_{e\in E}\frac{1}{2} = \frac{|E|}{2}$$

כיוון שכל קשת בפתרון האופטימלי היא קשת בגרף, בהכרח מתקיים $|E| \geq opt$. אנחנו קיבלנו כי בתוחלת מס' הקשתות הוא חצי, לכן $\frac{1}{2}|E| \geq \frac{1}{2}opt$. כלומר, האלגוריתם הפשוט הנ"ל מצליח בתוחלת לספק בממוצע כחצי מהפתרון האופטימלי. לכן מדובר באלגוריתם 0.5-approximation.

1.3.2 אלגוריתם חיפוש מקומי

כעת נציג אלגוריתם דטרמיניסטי פולינומי שמקרב את Max-cut. האלגוריתם פועל בגישה חמדנית - הוא מתחיל מחלוקה שרירותית של הקודקודים, וכל עוד ניתן לשפר את החתך ע"י העברת קודקוד יחיד מS ל$V\backslash S$ או להפך, הוא מבצע את ההעברה. כלומר, כל עוד קיים קודקוד שלא בS שהוספתו לS תגדיל את גודל החתך: הכנס אותו לS. אחרת, אם קיים קודקוד שהוצאתו מS תגדיל את גודל החתך, תוציא אותו מS. אחרת - לא ניתן לשפר: החזר את S.

Algorithm: Local Search for Max-Cut

1. $S \leftarrow$ arbitrary subset
2. While (true):
   (a) If $\exists v \notin S$ such that $|E(S\cup\{v\}, V\setminus(S\cup\{v\}))| > |E(S, V\setminus S)|$:
      i. $S \leftarrow S\cup\{v\}$
   (b) Else if $\exists v \in S$ such that $|E(S\setminus\{v\}, (V\setminus S)\cup\{v\})| > |E(S, V\setminus S)|$:
      i. $S \leftarrow S\setminus\{v\}$
   (c) Else: **return** S

ניתוח סיבוכיות זמן הריצה: בכל איטרציה של הלולאה, גודל החתך גדל באחד לפחות. מס' הקשתות המקסימלי בגרף לא מכוון עם n קודקודים חסום ע"י $|E| \leq \binom{n}{2}$. לכן, האלגוריתם יבצע לכל היותר $\binom{n}{2}$ איטרציות, כל אחת מהן בעלות פולינומית. לכן זמן הריצה פולינומי.

משפט 19. האלגוריתם הוא אלגוריתם *0.5-approximation* ל*Max Cut*.
הוכחה: נתבונן בתנאי העצירה באופטימום המקומי S. לכל קודקוד $v \in S$ מס' הקשתות הנכנסות לחתך לתוך v כלופר $|E(\{v\}, V\backslash S)|$ חייב להיות לפחות מחצית מדרגתו, שכן אחרת הוצאתו מ־S הייתה מגדילה את החתך בסתירה לכך שהאלגוריתם עצר. נסביר לעומק: בהינתן קודקוד כל‌שהו ל, סך הקשתות שיוצאות ממנו הן $deg(v)$. אלו מתחלקות לשני סוגים ־
1. קשתות שחוצות את החתך, מספרן $|E(\{v\}, V\backslash S)|$.
2. קשתות פנימיות בתוך S, מספרן הוא $|E(\{v\}, S)|$.
נניח ונעביר את v מהקבוצה S אל הקבוצה $V\backslash S$. במצב זה, הקשתות החוצות שהיו לו יפסיקו לחצות, והפנימיות יתחילו לחצות את החתך. השינוי בגודל החתך הרי יהיה:

$$\Delta = |E(\{v\}, S)| - |E(\{v\}, V\backslash S)|$$

האלגוריתם נעצר באופטימום מקומי, לכן מעבר שכזה בהכרח לא מגדיל את גודל החתך. לכן $\Delta \leq 0$. כלומר: $|E(\{v\}, S)| \leq |E(\{v\}, V\backslash S)|$. לכן נקבל: $|E(\{v\}, V\backslash S)| \geq \frac{\deg(v)}{2}$.
באופן זהה, לכל קודקוד $v \notin S$ מס' הקשתות ממנו אל S מקיים אף הוא $|E(\{v\}, S)| \geq \frac{deg(v)}{2}$.
אם נסכום את תרומת הקודקודים לחתך מעבר לכל v, כל קשת בחתך תספר בדיוק פעמיים (פעם לכל קודקוד בה). לכן גודל החתך הינו:

$$|E(S, V\backslash S)| = \frac{1}{2}\sum_{v\in V}|E(\{v\}, V\backslash S)| \geq \frac{1}{2}\sum_{v\in V}\frac{deg(v)}{2} = \frac{|E|}{2} \geq \frac{opt}{2}$$

לכן, גודל החתך בסיום הוא לפחות חצי מהפתרון האופטימלי. לכן $Max\text{-}Cut \in APX$.

הערה 20. החסם של האלגוריתם הדוק.

משפט 21. קיימים אלגוריתמי קירוב טובים יותר לחתך מקסימלי:
1. 1994: אלגוריתם קירוב שמשיג יחס קירוב של

$$\frac{\pi}{2} \cdot \min_{\theta\in[0,\pi]} \frac{\theta}{1-\cos(\theta)} \approx 0.87$$

2. משפט *Hasted*: תחת ההנחה כי $NP \neq P$, לא קיים אלגוריתם פולינומי שישיג יחס קירוב טוב יותר מ־$\frac{16}{17}$ לבעיה.

1.4 Minimum Makespan Scheduling

הגדרה 22. בעיית תזמון למזעור זמן הסיום מוגדרת באופן הבא: יהיו n משימות ־ $j_1, ..., j_n$ כך שלכל משימה j_i ישנו זמן עיבוד נדרש להשלמתה ־ j_i מייצג את הזמן שנדרש לבצע את המשימה הi. ויהי $m \in \mathbb{N}$ מס' המכונות אשר פועלות במקביל, ובקצב אחיד, עליהן ניתן להריץ את המשימות. המטרה: למצוא חלוקה/שיבוץ של n המשימות בין m המכונות כך שזמן הסיום הכולל יהיה מינימלי. זמן הסיום (makespan) מוגדר להיות זמן העבודה של המכונה העמוסה ביותר:

$$\text{Makespan} = \max_{1\leq k\leq m}\left(\sum_{j_i\in M_k} j_i\right)$$

משפט 23. בעיית תזמון למזעור זמן הסיום היא בעיה NP־קשה. גם כאשר $m = 2$, זו בעיית *Partition*, שגם היא NPC.

1.4.1 אלגוריתם חמדן

נציג אלגוריתם חמדן לקירוב הבעיה.

Greedy Algorithm for Job Scheduling

1. Order the jobs arbitrarily
2. Schedule jobs on machines in this order, scheduling the next job on the machine that has been assigned the **least amount** of work so far

האלגוריתם עובד באופן הבא: קובעים סדר שרירותי כלשהו עבור n המשימות, ואז עוברים על המשימות אחת אחת לפי הסדר שנקבע, ומשבצים את המשימה הנוכחית למכונה בעלת העומס המצטבר (!) הקטן ביותר באותו הרגע.

משפט 24. האלגוריתם מספק יחס קירוב של $2 - \frac{1}{m}$

הוכחה: ראשית, נשים לב לזוג אבחנות שחייבות להתקיים בכל פתרון אופטימלי. פתקיים בהכרח $opt \geq \frac{\sum_{i=1}^{n} j_i}{m}$ שכן סך העבודה היא סכום זמני המשימות, אם נחלק את העבודה הזו באופן מושלם בין m המכונות נקבל את מפוצץ העומס. המכונה העמוסה ביותר לא יכולה לסיים לפני המפוצץ הזה. לכן בהכרח, הפתרון האופטימלי הוא לפחות מפוצץ העומס. שנית, בהכרח מתקיים $opt \geq \max_i j_i$. שכן, מכונה שמקבלת את המשימה הארוכה ביותר חייבת לעבוד לפחות משך הזמן שלוקח לבצע אותה. כעת, נרצה להוכיח את יחס הקירוב.
תהי q המכונה שעבדה הכי הרבה זמן באלגוריתם שלנו, ויהי t_{max} זמן הסיום שלה. תהי j_s המשימה האחרונה ששובצה למכונה q. ויהי t_s זמן העבודה של המכונה q לפני ששובצה אליה המשימה האחרונה. מכאן זמן הריצה של האלגוריתם בהכרח:

$$alg\ time = t_s + j_s$$

נבחין: ברגע הזמן t_s בו המשימה j_s שובצה למכונה q, האלגוריתם החמדני בחר בה כי היא הייתה המכונה הפנויה ביותר. לכן, באותו הרגע כל m המכונות האחרות היו עמוסות לפחות באותה מידה ולכן כולן עבדו לפחות בזמן t_s. אם נסכום את עבודת המכונות עד לאותו הרגע $(m \cdot t_s)$, ונוסיף את המשימה האחרונה, נראה כי סכום זה חסום מלמטה ע"י סך כל המשימות שהרי לפי האבחנה שלנו חסום ב$m \cdot opt$. נקבל:

$$m \cdot t_s + j_s \leq \sum_{i=1}^{n} j_i \leq m \cdot opt$$

נחלק את אי השוויון בm ונקבל:

$$t_s + \frac{j_s}{m} \leq opt \Longrightarrow t_s \leq opt - \frac{j_s}{m}$$

נציב זאת בחזרה לביטוי זמן הסיום של האלגוריתם:

$$alg\ time = t_s + j_s \leq \left(opt - \frac{j_s}{m}\right) + j_s = opt + \left(1 - \frac{1}{m}\right) j_s$$

נשים לב שמהאבחנה שלנו מתקיים $j_s \leq opt$ בהכרח , לכן:

$$alg\ time \leq opt + \left(1 - \frac{1}{m}\right) opt = \left(2 - \frac{1}{m}\right) opt$$

כנדרש.

הערה 25. מדובר בחסם הדוק.

משפט 26. נניח אלגוריתם תזמון שבו ממיינים מראש את המשימות בסדר יורד לפי משך הביצוע:

$$j_1 \geq j_2 \geq ..... \geq j_n$$

אזי, האלגוריתם מבטיח $makespan$ שאינו גרוע יותר מ$\frac{3}{2}$ ביחס לפתרון האופטימלי.
הוכחה: יהי opt זמן הסיום המקסימלי בפתרון האופטימלי. נגדיר את הקבוצה

$$J_L = j_1, ..., j_k | \forall i : j_i > \frac{opt}{2}$$

כלומר, קבוצת כל המשימות שמשכן הוא לפחות חצי מזמן הסיום האופטימלי.
ראשית, נטען כי $k \leq m$, כלומר: מס' המשימות בקבוצה זו לא יכול לעלות על מספר המכונות. נניח בשלילה שזהו לא המצב, אזי $m < k$. לפי עקרון שובך היונים, אם יש יותר מm משימות גדולות ורק m מכונות, קיימת לפחות מכונה אחת שתקבל זוג מכונות גדולות. אך כל משימה בקבוצה זו גדולה ממש מ$\frac{opt}{2}$, לכן זמן העבודה של המכונה גדול ממש מopt בסתירה.
כעת, תהי q המכונה העמוסה ביותר שהגיעה לזמן הסיום $alg = t_{max}$. תהי j_s המשימה האחרונה ששובצה למכונה q, ושובצה אליה ברגע הזמן t_s. מכאן זמן סיום האלגוריתם הינו $alg = t_s + j_s$.
כעת, נחלק לשני מקרים ביחס למיקום האינדקס s:
- $s \leq k$: כלומר, המשימה האחרונה היא משימה "גדולה". כיוון שהמשימות ממוינות בסדר יורד, המשימה j_s בקידומת לפני או בזיוק k, אם מכונה q הייתה מקבלת משימה נוספת פרט לj_s גם אותה משימה הייתה שייכת לJ_L בעקבות המיון היורד, מה שהיה גורר לפחות 2 משימות גדולות על אותה מכונה, בסתירה כפי שראינו. לכן מכונה q מכילה אך ורק את המשימה j_s ומתקיים

$$alg = j_s \leq \frac{opt}{2} \leq \frac{3}{2} opt$$

- $s > k$. כלומר, המשימה האחרונה אינה משימה גדולה. כיוון ש$s > k$ המשמעות היא שj_s לא משימה גדולה ולכן $j_s \leq \frac{opt}{2}$. ברגע הזמן j_s בו שובצה המשימה j_s למכונה q, כל m המכונות היו פעילות ותפוסות - אחרת האלגוריתם החמדני היה בוחר במכונה פנויה. לכן, סך העבודה שבוצעה עד לזמן t_s חסום מלמטה ע"י העבודה של כל המכונות עד לאותו רגע, וכפי שראינו בהוכחה קודמת, זה חסום ע"י $m \cdot opt$. לכן:

$$t_s \cdot m \leq \sum_{i=1}^{n} j_i \leq m \cdot opt \Longrightarrow t_s \leq opt$$

לכן, נראה כי:

$$alg = t_s + j_s \leq \frac{1}{2} opt + opt = \frac{3}{2} opt$$

כפי שרצינו. כלומר, במקרה זה זמן העבודה עד להגעת המשימה האחרונה במכונה הכבדה ביותר הוא לכל היותר opt, והמשימה האחרונה חסומה ב$\frac{1}{2} opt$ בזמנה. כנדרש.

■

משפט 27. $(Graham, 1969)$. קיים אלגוריתם מתקדם יותר של תזמון למציאת $MakeSpan$ מינימלי, בו ממיינים את המשימות בסדר יורד לפי משך הביצוע שלהם, ומקבלים יחס קירוב של $\frac{4}{3}$ ביחס לפתרון האופטימלי. חסם זה הוא הדוק.

משפט 28. בעיית תזמון למציאת $Makespan$ מינימלי מקיימת סכמת קירוב בזמן פולינומי $(PTAS)$. המשמעות, לכל $\varepsilon > 0$ ניתן למצוא קירוב של $1 + \varepsilon$ בזמן ריצה פולינומי.

1.5 סכמות קירוב

הגדרה 29. PTAS (Polynomial-Time Approximation Scheme): בעיית אופטימיזציה מסוג מזעור מקיימת $PTAS$ אם לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן פולינומי.

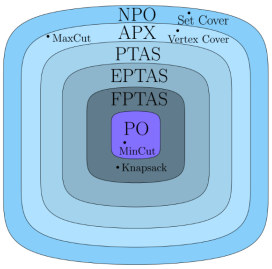
לדוגמה, זמן ריצה הגיוני לאלגוריתם שכזה הוא $n^{\mathcal{O}(\frac{1}{\varepsilon})}$ או $n^{2\cdot\mathcal{O}(\frac{1}{\varepsilon})}$. למעשה זה פולינומי אך עדיין תלוי בדיוק בחזקה ויכול להיות איטי מאוד.

הגדרה 30. EPTAS (Efficient Polynomial-Time Approximation Scheme): בעיית אופטימיזציה מסוג מזעור מקיימת $EPTAS$ אם לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן ריצה של $\mathrm{Poly}(n) \cdot f\left(\frac{1}{\varepsilon}\right)$, באשר f יכולה להיות כל פונקציה שהיא.

למשל, זמן ריצה הגיוני לאלגוריתם שכזה הוא $\mathrm{Poly}(n) \cdot 2^{\mathcal{O}(\frac{1}{\varepsilon})}$. כאן, זמן הריצה יותר יעיל.

הגדרה 31. FPTAS (Fully Polynomial-Time Approximation Scheme). בעיית אופטימיזציה מסוג מזעור מקיימת $FPTAS$ אם כל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן ריצה של $\mathrm{Poly}\left(n+\frac{1}{\varepsilon}\right)$.

הערה 32. תחת ההנחה כי $P \neq NP$, כך נראית תמונת העולם של המחלקות:



כפי שראינו, MinCut לפי משפט $MFMC$ שקול לזרימה מקסימלית בגרף. זו בעיית אופטימיזציה לה קיים אלגוריתם פולינומי יעיל (Dinic). המחלקות מקיימות הירכיה של הכלות, כאשר למשל Set Cover $\in NPO$, שכן ניתן לאמת פתרון חוקי. וכן, APX זו מחלקת הבעיות שקיים עבורן אלגוריתם שמבטיח יחס קבוע - למשל VC,MaxCut.

משפט 33. תחת ההנחה כי $P = NP$, כל המבנה קורס ומתקיים $NPO = PO$. כלומר, כל מחלקות הסיבוכיות באופטימיזציה מתמוטט וקורס למחלקה PO.

1.6 מרחבים מטריים (Metric space)

מרחב מטרי הוא מבנה מתמטי אשר מגדיר פורמלית את המושג מרחק בין איברים בקבוצה כלשהי.

הגדרה 34. מרחב מטרי הוא זוג סדור (X, d_X) באשר X היא קבוצה כלשהי וכן $d_X : X \times X \to \mathbb{R}_{\geq 0}$ פונקציית מרחק המקיימת שלוש אקסיומות:
1. זהות: לכל $x, y \in X$ מתקיים $d_X(x,y) = 0$ אם ורק אם $x = y$.
2. סימטריה: לכל $x, y \in X$ מתקיים $d_X(x,y) = d_X(y,x)$.
3. אי שוויון המשולש: לכל $x, y, z \in X$ מתקיים $d_X(x,y) \leq d_X(x,z) + d_X(z,y)$.

דוגמה 35. מס' דוגמאות למרחבים מטריים:

$$\ell_2 \in \mathbb{R}^d : d_{\ell_2}(x,y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

$$\ell_1 \in \mathbb{R}^d : d_{\ell_1}(x,y) = \sum_{i=1}^{d} |x_i - y_i|$$

$$d_G \text{ s.t } G=(V,E) \text{ is a graph} : d_G(u,v) = \min\{|P| \text{ P is a path from u to v}\}$$

הגדרה 36. יהי (X, d_X) מרחב מטרי, כדור סגור ברדיוס $r > 0$ סביב נקודה $x \in X$ מוגדר כקבוצת כל הנקודות במרחב שמרחקן מx הוא לכל היותר r. כלומר:

$$B_X(x,r) = \{y \in X | d_X(x,y) \leq r\}$$

הגדרה 37. בעיית מרכז הk. בהינתן מרחב מטרי (X, d_X) ופרמטר שלם $k \geq 1$, המטרה היא למצוא תת קבוצה $K \subseteq X$ המקיימת $|K| = k$ (כלומר, בחירת k נקודות מרכזיות), אשר תמזער את הביטוי:

$$\max_{x \in X} d_X(x,K) = \max_{x \in X} \min_{y \in K} d_X(x,y)$$

לצורך הבנת הרעיון, נדמיין שאנו צריכים להקים k תחנות כיבוי אשר בעיר. אנו רוצים למקם אותם כך שהתושב הכי רחוק מתחנה כלשהי לא יסבול מרחק עצום. כלומר, לוקחים את המרחק של כל נקודה למרחק הקרוב אליה ביותר (min), כלומר, זהו המרכז הקרוב y, ומחפשים את המרכז שסובל מהלקוח הרחוק ביותר ־ מנסים לבחור את המיקומים, כך שהמקסימום הזה יהיה מינימלי. כלומר: מזעור המקרה הגרוע ביותר עבור האיש הרחוק ביותר. באופן שקול נוכל להגדיר את הבעיה:
מציאת הרדיוס המינימלי r וקבוצת מרכזים K כך שאיחוד הכדורים ברדיוס r סביב כל נקודות המרכז מכסה את כל המרחב X:

$$X = \bigcup_{y \in K} B_X(y,r)$$

כלומר, כל המרחב X מכוסה לחלוטין ע"י איחוד הכדורים ברדיוס r אשר המרכז שלהם נמצא בK.

1.6.1 אלגוריתם גונזלס (Gonzalez Algorithm) לבעיית הk-center

האלגוריתם הוא אלגוריתם חמדן פשוט ויעיל שנבנה עבור הבעיה. בתחילה, בוחרים מרכז ראשון ־ t_1: באופן שרירותי, אותו מקבעים כמרכז הראשון. בכל איטרציה, בוחרים קודקוד t_{i+1} את הקודקוד שהוא הכי רחוק מבין כל המרכזים שנבחרו עד כה: $\{t_1, ..., t_i\}$ והופכים אותו למרכז הבא. כך ממשיכים עד שבוחרים k מרכזים. בסיום $K = \{t_1, ..., t_k\}$.

Algorithm: k-Center Selection (Farthest-First Traversal)

1. **Choose the first center (t_1):** Pick an arbitrary vertex in the space and set it as the first center
2. **Iterative selection (t_{i+1}):** At each step, choose the vertex that is **farthest** from all centers chosen so far ($\{t_1, \ldots, t_i\}$), and make it the next center
3. Continue until exactly k centers have been chosen: the final set of centers is $K = \{t_1, \ldots, t_k\}$

נגדיר את הפרמטר

$$\Delta = d_X(t_{k+1}, \{t_1, ..., t_k\})$$

זהו המרחק של הנקודה ה$k+1$ (הנקודה הבאה שהאלגוריתם היה בוחר אילו היינו ממשיכים עוד צעד אחד), כלומר ־ זהו המרחק של נקודה בודדת t_{k+1} שמוגדר להיות המרחק מהמרכז הקרוב ביותר אליה מבין המרכזים בקבוצה, אך מהגדרת האלגוריתם זו הנקודה הרחוקה ביותר מבין כל נקודות המרחב ביחס לקבוצה K ולכן

$$\Delta = \max_{x \in X} \min_{y \in K} d_X(x,y)$$

משפט 38. מתקיים $X = \bigcup_{y \in K} B_X(y, \Delta)$. כלומר, המרחב כולו מכוסה ע"י איחוד הכדורים ברדיוס Δ סביב איברי K.
הוכחה: תהי נקודה $x \notin K$, לפי אופן פעולת האלגוריתם, t_{k+1} נבחרה בתור הנקודה שהמרחק שלה מK הוא מקסימלי מבין כל הנקודות במרחב, לכן המרחק של כל נקודה כלשהי מK חייב להיות קטן או שווה למרחק של t_{k+1} מK כלומר

$$d_X(x, K) = d_X(x, \{t_1, ..., t_k\}) \leq d_X(t_{k+1}, \{t_1, ..., t_k\}) = \Delta$$

מכיוון שמרחקה של כל נקודה מx אל המרכז הקרוב ביותר אליה בK לכל היותר Δ נובע כי x נמצאת בתוך כדור ברדיוס Δ סביב אחד המרכזים, לכן כל המרחב מכוסה. כנדרש.

■

משפט 39. יהיו $i \neq j$ אינדקסים של זוג נקודות t_i, t_j מתוך $k+1$ הנקודות הראשונות שהאלגוריתם בחר. אזי,

$$d_X(t_i, t_j) \geq \Delta$$

הוכחה: נניח בה"כ כי $i < j$. אזי, t_i נבחרה לפני t_j. כיוון שt_j עצמה נבחרה להיות הרחוקה ביותר מהקבוצה הקודמת $\{t_1, ..., t_{j-1}\}$ והנקודה t_i הרי נמצאת בקבוצה זו, המרחק של t_j מקבוצה זו חסום מלמטה ע"י המרחק הספציפי שלה מt_i לכן $*$. מכאן:

$$d_X(t_i, t_j) \geq_* d_X(t_j, \{t_1, ..., t_{j-1}\}) \geq_{**} d_X(t_{k+1}, \{t_1, ..., t_{j-1}\}) \geq_{***} d_X(t_{k+1}, \{t_1, ..., t_k\}) = \Delta$$

באשר $**$ נכון כי האלגוריתם בחר את t_j בתור הנקודה הרחוקה ביותר במרחב כולו מהקבוצה $\{t_1, ..., t_{j-1}\}$. בפרט, כיוון שt_{k+1} היא גם נקודה במרחב, המרחק שלה מהקבוצה היה קטן־שווה.
וכן $***$ נכון כי הוספת נקודות לקבוצה יכולות רק להקטין את המרחק של t_{k+1} מהקבוצה.

הערה 40. הנקודה ה$k+1$ לא נמצאת מעולם ע"י האלגוריתם, אלא משמשת לניתוח מתמטי בלבד של החסם.

משפט 41. מתקיים $opt \geq \frac{\Delta}{2}$ ולכן האלגוריתם של גונזלס הוא *2-approximation* לבעיית מרכז הk.
הוכחה: נסתכל על פתרון אופטימלי כלשהו לבעיה, נסמנו K_{opt}: הוא מכיל k מרכזים. נגדיר פונקציה f אשר משייכת כל אחת מ$k+1$ הנקודות שבחרנו $(\{t_1, ..., t_{k+1}\})$ אל המרכז הקרוב ביותר אליה מתוך הפתרון האופטימלי. לפי עקרון שובך היונים, ישנם $k+1$ נקודות מקור ורק k יעדים, לכן חייבת להיות לפחות זוג נקודות בסדרה שלנו, נסמנם t_i, t_j שמשויכות לאותו מרכז אופטימלי בדיוק. נסמנו y. כלומר:

$$f(t_i) = f(t_j) = y$$

כעת, לפי אי שוויון המשולש עבור הנקודות הללו מתקיים

$$d_X(t_i, t_j) \leq d_X(t_i, y) + d_X(y, t_j)$$

אז, אנו יודעים כי המרחק בין זוג הנקודות הוא לפחות Δ לפי משפט קודם. לכן:

$$\Delta \leq d_X(t_i, y) + d_X(y, t_j)$$

כיוון שy הוא מרכז בפתרון האופטימלי, והמרחק של כל נקודה למרכז הקרוב ביותר אליה מוגבל ע"י רדיוס האופטימום, מתקיים $d_X(t_i, y), d_X(y, t_j) \leq opt$. נקבל:

$$\Delta \leq 2opt \Longrightarrow \frac{\Delta}{2} \leq opt$$

אם כן, האלגוריתם של גונזלס מייצר פתרון בו הרדיוס המקסימלי הוא בדיוק Δ, לכן $\Delta \leq 2opt$ כפי שראינו, כלומר האלגוריתם מחזיר רדיוס Δ וקבוצת מרכזים שהרדיוס הוא לכל היותר 2 מהפתרון האופטימלי ולכן מדובר ב*2-approximation*.
■

משפט 42. תחת ההנחה כי $P \neq NP$, לכל $\varepsilon > 0$ לא קיים אלגוריתם קירוב בזמן פולינומי ביחס של $2 - \varepsilon$ לבעיית ה$k - center$.
הוכחה: ההוכחה מתבצעת ברדוקצייה מDS. נזכר כי DS מוגדרת להיות: בהינתן גרף $G = (V, E)$ קבוצה $A \subseteq V$ נקראת קבוצה שולטת אם לכל $v \in V$ מתקיים $v \in A \ or \ \exists w \in \Gamma(v) : w \in A$. בעיית ההחלטה המתאימה, היא האם בהינתן $k \in \mathbb{N}$ קיימת קבוצה שולטת בגרף בגודל לכל היותר k. נגדיר את הרדוקציה באופן הבא:
בהינתן גרף $G = (V, E)$ וערך k טבעי, נגדיר מרחב מטרי על קודקודי הגרף באמצעות מטריקת המסלולים הקצרים ביותר בין קודקודי הגרף. כלומר - (V, d_G).
כעת, נניח בשלילה שקיים אלגוריתם קירוב בזמן פולינומי ביחס $t = 2 - \varepsilon$ עבור $\varepsilon > 0$ לבעיית הK-*center*. נפעיל את האלגוריתם על המרחב (V, d_G) כדי לקבל פתרון $K \subseteq V$ בעל רדיוס α. כלל ההחלטה יוגדר להיות: אם $\alpha < 2$ החזר כן, אחרת אם $\alpha \geq 2$ החזר לא. נוכיח נכונות:
מקרה ראשון - קיימת קבוצה שולטת בגודל לכל היותר k: משמעות הדבר כי לכל קודקוד בגרף יש שכן בקבוצה זו, או שהוא בעצמו בה. לכן, מרחב הקודקודים מכוסה לחלוטין ע"י איחוד כדורים ברדיוס 1 סביב איברי K כלומר $V = \bigcup_{u \in K} B_G(u, 1)$, לכן קיים פתרון מדויק אופטימלי לבעיית הK-*center*. כיוון שהנחנו שאלגוריתם הקירוב בעל יחס של $t = 2 - \varepsilon$, הרדיוס α שהאלגוריתם יחזיר הינו לכל היותר עם יחס השגיאה

$$\alpha \leq 1 \cdot t = 2 - \varepsilon < 2$$

ולכן נשיב כן במקרה זה.
מקרה שני - כל קבוצה שולטת בגרף היא מגודל $k+1$ ומעלה. במקרה זה לא קיים כיסוי בגודל k, לכן כל פתרון לבעיית הK-*center* חייב להיות בעל רדיוס לפחות 2. כלומר $opt \geq 2$. כיוון שהאופטימום הוא לפחות 2, ואלגוריתם הקירוב מבטיח תוצאה שאינה גרועה פי יותר מt ביחס לאופטימום המינימלי, הערך α שהאלגוריתם יפצא יקיים $\alpha \geq 2$ בהכרח. לכן, האלגוריתם ישיב לא: בצדק.
סה"כ, אם היה קיים אלגוריתם פולינומי ביחס של $2 - \varepsilon$ לבעיית *K-center*, היינו יכולים להכריע בזמן פולינומי את DS, אז תחת ההנחה $P \neq NP$, לא יתכן הדבר. כנדרש.
■

1.7 Bin Packing Problem

הגדרה 43. בהינתן קבוצה סופית של פריטים $U=\{u_1,....,u_n\}$ עם גדלים רציונליים, כלומר $\forall i: s(u_i)\in[0,1]$. הפלט הוא חלוקה של U לתתי קבוצות־ מכלים, $U_1,....,U_k$ כך שלכל $j\in\{1,...,k\}$ סכום הגדלים של הפריטים בתוכו לא עולה על 1, כלומר:

$$\forall j\in\{1,...,k\}:\sum_{u_i\in U_j}s(u_i)\leq 1$$

כאשר המטרה היא מזעור $k\in\mathbb{N}$. מדובר בבעיה שהיא NPC.

מדובר בבעיה שקשה לקרב, תחת ההנחה $P\neq NP$ כמובן.

משפט 44. אם $P\neq NP$, לכל $\varepsilon>0$ לא קיים אלגוריתם קירוב ביחס של $\frac{3}{2}-\varepsilon$ לבעיית ה*Bin-Packing*.
הוכחה: בדומה להוכחה הקודמת שראינו, ההוכחה תתבצע ברדוקציה מ*Partition* - נזכר כי בעיה זו מוגדרת בהינתן קבוצה $X=\{x_1,....,x_n\}$ האם ניתן לחלק את X לשתי קבוצות שסכומן שווה.
נניח בשלילה שקיים אלגוריתם קירוב ביחס של $\frac{3}{2}-\varepsilon$ לבעיית ה*Bin-Packing*. בהינתן קלט $S=\{x_1,.....,x_n\}$ ל*Partition* נסמן $\mu=\sum_{x_i\in S}x_i$ ונבנה קלט עבור *Bin-Packing* באופן הבא:

$$S'=\left\{\frac{2}{\mu}x_1,\frac{2}{\mu}x_2,.....,\frac{2}{\mu}x_n\right\}$$

נבחין כי:

$$\sum_{x_i\in S'}\frac{2}{\mu}x_i=\frac{2}{\mu}\sum_{x_i\in S}x_i=\frac{2}{\mu}\cdot\mu=2$$

כלומר, סכום האיברים בקבוצה שבנינו הוא בדיוק 2. נפעיל את אלגוריתם הקירוב בהנחה שהוא פועל על הקלט החדש S', האלגוריתם יחזיר פתרון אריזה הכולל מס' כולל של k מכלים. אם מס' הכלים שהתקבל הוא בדיוק $k=2$ נחזיר כן, אחרת כלומר $k\geq 3$ נחזיר לא.
כעת נרצה להוכיח את נכונות הרדוקציה:
בכיוון ראשון, נניח שקיימת חלוקה חוקית בבעיית החלוקה המקורית. אזי לכל קבוצה בחלוקה ישנו סכום של $\frac{\mu}{2}$, משמעות הדבר במרחב של S' כי ניתן לארוז את כל הפריטים בדיוק בזוג מכלים. כלומר, $opt=2$. האלגוריתם מבטיח יחס קירוב של $\frac{3}{2}-\varepsilon$ ולכן מס' הכלים שהאלגוריתם יחזיר יקיים את החסם:

$$k<\left(\frac{3}{2}-\varepsilon\right)opt=3-2\varepsilon$$

מס' המכלים חייב להיות מס' שלם, ולכן בהכרח נקבל $k\leq 2$ ולכן האלגוריתם ישיב כן.
בכיוון שני, נניח שלא קיימת חלוקה בבעיית ה*Partition* המקורית. משמעות הדבר כי שום חלוקה של המספרים לזוג קבוצות לא תניב סכומים ששווים בדיוק ל$\frac{\mu}{2}$. כלומר, לא אפשרי לארוז את הפריטים של S' (שסכומם הכולל הוא 2) בשני מכלים בלבד, כך שכל מיכל יכיל לכל היותר 1 כי אם היו זוג מכלים כאלו, הם בהכרח היו חייבים להיות מלאים עד בדיוק 1, ואז היינו מקבלים זוג מכלים מלאים בסתירה לכך שאין פתרון לבעיית החלוקה. לכן בהכרח $Opt\geq 3$, האופטימום הוא לפחות 3 ולכן אלגוריתם הקירוב לא יוכל להחזיר 2 שהרי יחזיר ערך גדול או שווה מ3, ולכן נחזיר לא.
לכן סה"כ הרדוקציה חוקית ונכונה, אם היה קיים אלגוריתם כנ"ל בזמן פולינומי היינו מכריעים את *Partition* בסתירה לכך ש$P\neq NP$.

■

1.7.1 First fit alogorithm

כעת נציג אלגו' פשוט שכן יקרב את Bin-Packing, פולינומי, שעובד באופן הבא: לכל פריט μ_i לפי סדר הופעתו, הכנס אותו אל המיכל בעל האינדקס הקטן ביותר שיכול להכיל אותו.

Algorithm: First Fit (Bin Packing)

The First Fit approximation algorithm for the bin packing problem works as follows:

1. We start with n empty bins: $B_1, B_2, \ldots, B_n$
2. For each item μ_i, in order of appearance ($i \in \{1, \ldots, n\}$), we insert it into the bin with the **smallest index** that it can still fit into (i.e. the sum of weights in that bin does not exceed 1 after the insertion)

משפט 45. מספר המכלים שאלגוריתם ה*First-Fit* מחזיר עבור קלט I, יסומן ב$FF(I)$ ויקיים:

$$FF(I) \leq 2 \cdot opt + 1$$

ולכן, מדובר באלגוריתם *3-approximation*.
הוכחה:נשים לב כי המשקל הכולל של כל הפריטים מוכל תחת הפתרון האופטימלי, כיוון שכל מיכל בפתרון האופטימלי מכיל משקל כולל של לכל היותר 1 ולכן מתקיים

$$Opt \geq \sum_{\mu_i \in U_j} s(\mu_i)$$

כלומר, המשקל הכולל חסם תחתון לפתרון האופטימלי. כעת, יהיו $B_1, ..., B_m$ המכלים שהוחזרו ע"י האלגוריתם. נטען: יש לכל היותר מיכל יחיד B_j שמשקלו קטן או שווה מחצי. שכן, אילו היו שני מיכלים שונים שמשקלם קטן מחצי, האלגוריתם היה מאחד אותם. כיוון שיש לפחות $m-1$ מכלים שמשקל כל אחד מהם גדול ממש מ$\frac{1}{2}$, סכום המשקלים הכולל של כל הפריטים מקיים

$$Opt \geq \sum_{\mu_i \in U_j} s(\mu_i) \geq \frac{m-1}{2} \Longrightarrow Opt \geq \frac{m-1}{2} \Longleftrightarrow 2 \cdot Opt \geq m-1 \Longrightarrow m \leq 2 \cdot Opt + 1$$

נשים לב כי $opt, \geq 1$, לכן נוכל לומר כי $m \leq 2 \cdot opt + 1 \leq 2 \cdot opt + opt = 3 \cdot opt$, ולכן האלגוריתם היוו *3-approximation*.
■

משפט 46. לכל קלט I, אלגוריתם ה*First-Fit* מקיים את החסם:

$$FF(I) \leq \frac{17}{10} \cdot opt(I) + 2$$

משפט 47. אלגוריתם הFFD מוגדר באופן דומה לFF, כאשר הוא מתחיל במיון הפריטים בסדר יורד - מהגדול לקטן. עבור אלגוריתם זה, מתקיים לכל קלט I:

$$FFD(I) \leq \frac{11}{9} \cdot opt(I) + 4$$

כלומר, הוא קרוב מאוד (≈ 1.22) לפתרון האופטימלי במקרה הגרוע ביותר, ולכן הוא מאלגוריתמי הקירוב היעילים והפופולריים ביותר לבעיה.

1.7.2 PTAS עבור Bin-Packing

משפט 48. לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם בזמן פולינומי שמוצא פתרון ל*Bin-Packing* המשתמש בלכל היותר

$$(1+\varepsilon)\, Opt + 1 \ bins$$

הוכחת המשפט, והאלגוריתם עצמו, מורכבים. לכן, נחלק את הבעיה למס' מקרים פרטיים, ובסיום נאחד אותם לאלגוריתם אחד כולל.

משפט 49. נניח כי כל הפריטים הם בגודל לכל היותר δ, אזי מס' המכלים של אלגוריתם FF מקיים

$$FF\left(I\right)\leq\left(1+2\delta\right)\cdot opt+1$$

הוכחה: אם $\delta\geq\frac{1}{2}$, נקבל כי $1+2\delta\geq2$ ולכן זה בדיוק הטענה שראינו קודם. אחרת, נניח $\delta<\frac{1}{2}$.
נשים לב: כל המכלים פרט לאחרון $B_1,....,B_m$ מכילים משקל של לפחות $1-\delta$. נניח בשלילה שקיים מיכל כלשהו B_j כך ש$j<m$ ־ מיכל שאינו האחרון, בו סכום המשקלים קטן ממש מ$1-\delta$. אם סכום המשקלים בB_j קטן ממש מ$1-\delta$, אזי, המקום הפנוי במיכל הוא לפחות: $1-\left(1-\delta\right)=\delta$, אך כל הפריטים הם בגודל לכל היותר δ: זו סתירה, שכן אחרת היה נכנס לפיכל זה פריט כלשהו קודם.
לכן, ישנם $m-1$ מיכלים, בכל אחד מהם יש לפחות $1-\delta$ משקל, מכאן שסכום המשקלים שנמצאים במכלים הללו הוא לפחות

$$\left(FF(I)-1\right)\cdot\left(1-\delta\right)\leq opt$$

שכן ברור שסכום המשקלים חייב להיות חסם תחתון לפתרון האופטימלי, שהוא מס' הסלים, נקבל כי

$$FF(I)-1\leq\frac{OPT}{1-\delta}\Longrightarrow FF(I)\leq\frac{OPT}{1-\delta}+1$$

אך, כיוון ש$\delta<\frac{1}{2}$, אזי $\frac{1}{1-\delta}\leq2\delta+1$ וקיבלנו את הדרוש.

משפט 50. כעת נניח כי ישנה קבוצה של k ערכים אפשריים בלבד. נסמן $A=\{\alpha_1,.....,\alpha_k\}$. במקרה זה, ניתן למצוא את הפתרון האופטימלי בזמן פולינומי של $\mathcal{O}\left(n^{2k+1}\right)$ באמצעות תכנון דינמי.
הוכחה: על מנת להשתמש בתכנון דינמי, נצטרך להגדיר מהו מצב. בבעיה שלנו, מצב מתאר אילו פריטים נמצאים באריזה שלנו.
כיוון שיש רק k סוגי גדלים שונים, אנו לא חייבים לרשום את כל הרשימה של הפריטים. אלא, מספיק שנרשום כמה פריטים יש מכל סוג. לכן כל מצב ייוצג ע"י וקטור בן k איברים

$$\mathcal{S}=\left\{\left(\beta_1,.....,\beta_k\right)\in\mathbb{N}^k|\sum_{i=1}^{k}\beta_i\leq n\right\}$$

באשר β_i הם מס' הפריטים בגודל α_i. נשים לב כי גודל מרחב המצבים הוא $\mathcal{O}\left(n^k\right)$. שכן, מדובר בבעיה קומבינטורית של חלוקת n כדורים לk תאים ופתרונה נתון ע"י

$$|\mathcal{S}|=\binom{n+k}{k}=\mathcal{O}\left(n^k\right)$$

כיוון שk קבוע, מדובר בגודל מרחב מצבים פולינומי. האלגוריתם עובד כאלגוריתם תכנון דינמי, בשלב הבסיס S_1 נמצא את כל תתי הקבוצות שניתן לארוז במיכל יחיד באופן חוקי , ניתן לחשב זאת ע"י מעבר על כל האפשרויות.
בצעד אינדוקטיבי: נניח שבידינו $\mathcal{S}_{i-1}$, כלומר אוסף כל המולטי־סטים שניתן לארוז ב$i-1$ מכלים, כדי למצוא את $\mathcal{S}_i$ נעזר באבחנה

$$\mathcal{S}_i\subseteq\{B+C|B\in\mathcal{S}_{i-1},C\in\mathcal{S}_1\ \ and\ |B|+|C|\leq n\}$$

כלומר, לקיחת מולטי סט שארזנו ב$i-1$ מכלים והוספת מיכל יחיד מלא.
זמן הריצה: כדי לחשב את $\mathcal{S}_i$ לכל שלב i, אנו עוברים על כל הזוגיות האפשריים במרחב המצבים בעלות: $\mathcal{O}\left(n^k\right)\cdot\mathcal{O}\left(n^k\right)=\mathcal{O}\left(n^{2k}\right)$.
יש סה"כ n מכלים אפשריים שהרי $1\leq i\leq n$ ולכן זמן הריצה הכולל הינו

$$\mathcal{O}\left(n\right)\cdot\mathcal{O}\left(n^{2k}\right)=\mathcal{O}\left(n^{2k+1}\right)$$

משפט 51. נניח כעת כי כל הפריטים בקלט הם בגודל של לפחות δ. כלומר לכל פריט μ_i מתקיים $s(\mu_i) \geq \delta$. אזי, קיים אלגוריתם שמייצר פתרון לבעיית אריזת המכלים המקיים:

$$Alg(U) \leq (1+\delta) \cdot Opt(U) + 1$$

וזמן הריצה של האלגוריתם הינו $\mathcal{O}\left(n^{\frac{2}{\delta^2}+1}\right)$.

הוכחה: יהי k כלשהו, אשר ערכו יקבע בהמשך הניתוח. נגדיר את האלגוריתם הבא:

- ניקח את קבוצת הפריטים U ונמיין אותם לפי משקלם.
- נחלק את הפרטים הממויינים ל$\left[\frac{n}{k}\right]$ קבוצות ונסמן $G_1, ..., G_{\left[\frac{n}{k}\right]}$.
- כל קבוצה G_i מכילה בדיוק k פריטים - פרט לאחרונה, שיכולה להכיל פחות פריטים.
- הקבוצות מסודרות לפי סדר גדלים, הקבוצה G_1 מכילה את הפרטים הגדולים ביותר וכו'.

על מנת לפתור את הבעיה ביעילות, נגדיר מופע קטן יותר U' באופן הבא:

1. נתעלם לחלוטין מהקבוצה הראשונה G_1.
2. לכל שאר הקבוצות $2 \leq i \leq \left[\frac{n}{k}\right]$ נשנה את המשקל של כל הפריטים בקבוצה כך שיהיו שווים למשקל הפריט הכבד ביותר באותה הקבוצה.

נטען כמה טענות. ראשית, מתקיים $Opt(U') \leq Opt(U)$: שכן, קיימת פונקציה חד חד ערכית $f : U' \to U$ כך שכל אלמנט במופע החדש U' ממופה לאלמנט במופע המקורי שלו במשקל שווה או גדול יותר, כיוון שכל פריט בU' הוחלף למשקל המקסימלי של קבוצתו, ובמופע המקורי U היו שם פריטים מקוריים, כל אריזה חוקית בU יכולה לשמש בסיס לאריזה בU', לכן סה"כ האופטימום של U' לא יכול לעלות על האופטימום של U.

שנית, ניתן לחשב את $Opt(U')$ בזמן $\mathcal{O}\left(n^{2\cdot\frac{n}{k}+1}\right)$. שכן: נשים לב כי במופע החדש U' כיוון שזרקנו את G_1 ישנן $\left[\frac{n}{k}\right]-1$ קבוצות, בכל קבוצה כזו הפריטים קיבלו ערך אחיד ולכן ישנם לכל היותר $\frac{n}{k} \leq \left[\frac{n}{k}\right]-1$ משקלים שונים סה"כ בתוך U'. כפי שראינו, נקבל בדיוק את הטענה הקודמת בשימוש בתכנות דינמי.

כעת, לאחר שהצלחנו לפתור את U' נרצה לפתור את המופע המקורי U. האלגוריתם יורכב משלושת השלבים הבאים:

1. לוקחים כל פריט ופריט בקבוצה G_1 ושמים אותו במיכל נפרד משל עצמו. סה"כ, ישנם k מכלים.
2. פותרים את הפתרון האופטימלי למופע הקטן U'.
3. משתמשים במבנה האריזה האופטימלי של U' על מנת לארוז חזרה את יתרת הקבוצות $G_2, ..., G_{\left[\frac{n}{k}\right]}$ במופע המקורי U. נשים לב כי אריזה זו חוקית גם אצלנו שכן כל משקל בבעיה U' גדול או שווה למשקל בU.

כעת, מס' המכלים הכולל של האלגוריתם: השתמשנו בk מכלים עבור G_1 ובעוד $Opt(U')$ בשלב האחרון. לכן סה"כ

$$Alg(U) = Opt(U') + k$$

תחת האבחנה $Opt(U') \leq Opt(U)$ נקבל $Alg(U) \leq Opt(U) + k$. כעת, נגדיר את k להיות:

$$k = \left[\delta \cdot \sum_{\mu_i \in U} s(\mu_i)\right]$$

נשים לב כי $\sum_{\mu_i \in U} s(\mu_i) \leq Opt(U)$. לכן:

$$k \leq [\delta \cdot Opt(U)] \leq \delta \cdot Opt(U) + 1$$

ומכאן:

$$Alg(U) \leq Opt(U) + k \leq Opt(U) + \delta\, Opt(U) + 1 = (1+\delta)\, Opt(U) + 1$$

כפי שרצינו, כלומר יחס הקירוב כפי שרצינו. כעת, באשר לזמן הריצה: נזכר כי זמן הריצה הינו $\mathcal{O}\left(n^{2\cdot\frac{n}{k}+1}\right)$. נציב את הערך של k ונעזר באבחנה שכל הפריטים בגודל לפחות δ ונקבל

$$\frac{n}{k} = \frac{n}{[\delta \cdot \sum s(\mu_i)]} \leq \frac{n}{\delta \cdot n \cdot \delta} = \frac{1}{\delta^2}$$

ולכן זמן הריצה היגו:

$$\mathcal{O}\left(n^{\frac{2}{\delta^2}+1}\right)$$

כנדרש.

■

כעת, לאחר הוכחת כל הלמות: נתפנה להצגת האלגוריתם הכללי.

Algorithm: General Case Bin Packing

1. Let U_L be all the items in U of weight $\geq \frac{\varepsilon}{2}$, and $U_S = U \setminus U_L$
2. Run the algorithm of Lemma 2 on U_L
3. Use FF to pack U_S into remaining spaces of large item bins. When all such space is used, open new bins using FF

האלגוריתם משלב את זוג הלמות שכבר הוכחנו, יחד עם אלגוריתם הFF. מגדירים סף של $\delta = \frac{\varepsilon}{2}$, ומחלקים את קבוצת הפריטים לזוג קבוצות. אלו שגדולים או שווים לסף, ואלו שמתחתיו. לקבוצת הפריטים הגדולים, משתמשים באלגוריתם שהוכחנו בלמה לעיל. עבור הפריטים הקטנים, מנסים לארוז אותם באמצעות אלגוריתם FF אל המקום הפנוי שנותר במיכלים הקיימים של הפרטים הגדולים. אם נגמר המקום במיכלים הקיימים, פותחים מכלים חדשים בהתאם. כעת נרצה להוכיח שהאלגוריתם עושה את הדרוש, כלומר מניב יחס קירוב של $1+\varepsilon$ לכל $\varepsilon > 0$.

מקרה ראשון: לא נפתחו מיכלים חדשים במהלך הFF. אם כן, אם כל הפריטים הקטנים בU_S נכנסנו לרווחים הפנויים במיכלים הקיימים, אזי בהכרח

$$\mathrm{Alg}\left(U\right) = \mathrm{Alg}\left(U_L\right)$$

נפעיל את החסם של הלמה לעיל עבור קבוצת הפריטים הגדולים וכן $\delta = \frac{\varepsilon}{2}$ ונקבל:

$$\mathrm{Alg}\left(U_L\right) \leq \left(1+\frac{\varepsilon}{2}\right) \cdot \mathrm{Opt}\left(U_L\right) + 1$$

הרי בהכרח האופטימום של U_L חוסם מלמטה את האופטימום של U ונקבל:

$$\mathrm{Alg}\left(U_L\right) \leq \left(1+\frac{\varepsilon}{2}\right) \cdot \mathrm{Opt}\left(U\right) + 1 \leq (1+\varepsilon) \cdot \mathrm{Opt}\left(U\right) + 1$$

מקרה שני: כן נפתחו מיכלים חדשים במהלך הFF. נניח כי האלגוריתם פתח סה"כ m מיכלים חדשים עבור תהליך השיבוץ הסופי. נסתכל על $m-1$ המיכלים הראשונים $(B_1, ..., B_{m-1})$ ועל המיכל האחרון B_m. כיוון שאלגוריתם FF פועל בצורה חמדנית, כל המיכלים פרט למיכל האחרון חייבים להיות מלאים מעבר לסף מסוים. כיוון שהפרטים הקטנים בגודל קטן מ$\frac{\varepsilon}{2}$, המשקל הכולל בכל אחד מהכלים הללו פרט לאחרון חייב להיות לפחות $1-\frac{\varepsilon}{2}$, אחרת פריט היה נכנס לשם. לכן סכום המשקלים של כל הפריטים שנארזו ב$m-1$ המכלים הראשונים מקיים

$$(FF-1) \cdot \left(1-\frac{\varepsilon}{2}\right) < \text{Total Weight} \leq opt$$

$$\implies FF < \frac{\mathrm{OPT}}{1-\frac{\varepsilon}{2}} + 1 \leq (1+\varepsilon) \cdot \mathrm{OPT} + 1$$

וסיימנו.

אם כן, ננסה לחשוב מדוע האלגוריתם המורכב הזה עובד - קצת לאינטואיציה. האלגוריתם הנ"ל מבצע חלוקת עבודה. הפריטים הגדולים הם אלו שלרוב קובעים את מבנה הבעיה והאופטימום, אך יש מהם מעט יחסית. לכן, ניתן להשקיע זמן בלפתור אותם בצורה מדויקת - אך יקרה מבלי לפוצץ את סיבוכיות הזמן הכוללת. הפריטים הקטנים, אמנם רבים אך בגלל שהם קטנים מאוד יחסית, הם מתנהגים כמו חומר מילוי והם גמישים - אפשר פשוט לרוב לזרוק אותם לחורים בצורה חמדנית (FF).

1.8 Minimum Multiway Cut

הגדרה 52. בהינתן גרף $G=(V,E)$ ממושקל עם פונקציית משקל w, וקבוצה של צמתים מיוחדים הנקראים טרמינלים ומסומנת ב$K=\{v_1,...,v_k\}\subseteq V$.
חיתוך רב כיווני הוא תת קבוצה של צלעות $F\subseteq E$ כך שכאשר מסירים אותן ונוצר הגרף $G'=(V,E\backslash F,w)$, כל טרמינל $v_i\in K$ שייך לרכיב קשירות שונה.
המטרה: מציאת חיתוך רב כיווני F כך שמשקל הצלעות בחיתוך F מינימלי.
באופן חלופי, נוכל להגדיר את הבעיה כיצירת חלוקה של הצמתים V לקבוצות $S_1,....,S_k$ כך שלכל i $v_i\in S_i$ והמטרה היא מזעור

$$\sum_{i<j} w\left(E\left(S_i,S_j\right)\right)$$

לכל $k\geq 3$, מדובר בבעיה NP־קשה.

נציג את האלגוריתם הבא:

Algorithm: Multiway Cut

1. For every $v_i\in K$, let E_i be the minimum $(v_i,K\setminus\{v_i\})$ cut
2. Let E_j be the cut of maximum weight among $E_1,\ldots,E_k$
3. **return** $\bigcup_{i\in[1,k]\setminus\{j\}} E_i$

האלגוריתם פועל באופן הבא ־ לכל טרמינל $v_i\in K$, האלגוריתם מוצא באמצעות רדוקציה לזרימה מקסימלית בזמן פולינומי את החיתוך המינימלי $(v_i,K\backslash\{v_i\})$ ומסמנו בE_i. בהמשך, האלגוריתם מוצא את החיתוך E_j בעל המשקל המקסימלי מבין החיתוכים $E_1,....,E_k$ ומחזיר את איחוד כל החיתוכים, פרט לאותו E_j. נשים לב שהאלגוריתם בהכרח מחזיר פתרון חוקי, כלומר לכל זוג טרמינלים הם נמצאים ברכיב קשירות אחר. שכן, בשלב הראשון האלגוריתם מוצא לכל טרמינל v_i חיתוך E_i שמנתק אותו משאר הטרמינלים בגרף ־ בפועל, האלגוריתם מחזיר את איחוד כל החיתוכים פרט לאחד. נבחין שזה מבטיח שכל זוג טרמינלים מנותק. שכן: יהיו v_i,v_q זוג טרמינלים. כיוון שהשמטנו מהאיחוד רק את E_j, לפחות אחד מהשניים ־ E_i או E_q נמצא בוודאות בתוך קבוצת הצלעות שהשבנו. בה"כ E_i נכלל בתוצאה F: לפי הגדרה E_i הוא חיתוך שמנתק את v_i משאר הטרמינלים, בפרט הוא מנתק אותו מv_q שנמצא בקבוצת האחרים ־ לכן אין מסלול מv_i לv_q. הערה ־ נשים לב כי F זו קבוצה של צלעות שנמחקת מהגרף. כיוון שמדובר באיחוד של חתכים, בהכרח כל טרמינל יהיה מנותק מאחרים.

משפט 53. האלגוריתם מספק יחס קירוב של $2-\frac{2}{k}$.
הוכחה: יהי פתרון אופטימלי כלשהו לבעיה, E^*. לכל טרמינל v_i, נגדיר את S_i להיות רכיב הקשירות של v_i בגרף שנוצר לאחר הסרת צלעות הפתרון האופטימלי. כלומר, בגרף $G'=(V,E\backslash E^*,w)$.
נגדיר את $E_i^*=E\left(S_i,V\backslash S_i\right)$ להיות קבוצת הצלעות היוצאות החוצה מהרכיב של v_i בפתרון האופטימלי. כיוון שכל הטרמינלים האחרים אינם נמצאים ברכיב הקשירות S_i, קבוצת הצלעות E_i^* מהווה חיתוך חוקי מהסוג $(v_i,K\backslash\{v_i\})$. כיוון שE_i מראש הוגדר כחיתוך המינימלי האפשרי המנתק את v_i משאר הטרמינלים, נובע כי בהכרח משקלו קטן או שווה למשקל החיצוני בפתרון האופטימלי:

$$w\left(E_i\right)\leq w\left(E_i^*\right)$$

עבור כל הטרמינלים, נקבל כי:

$$\sum_{i=1}^{k}w\left(E_i\right)\leq\sum_{i=1}^{k}w\left(E_i^*\right)=_*2w\left(E^*\right)$$

באשר $*$ נכון כי כל צלע שנמצאת בפתרון האופטימלי E^* בהכרח מחברת בין שני רכיבי קשירות שונים S_i,S_j ולכן צלע זו תופיע פעמיים בסכום הE_i^*. מכאן נקבל:

$$\sum_{i=1}^{k}w\left(E_i\right)\leq2w\left(E^*\right)$$

האלגוריתם מייצר את הפתרון הסופי F ע"י איחוד הקבוצות פרט לחיתוך הכבד ביותר E_j

$$w\left(F\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-w\left(E_j\right)$$

נראה כי כיוון שE_j נבחר להיות החיתוך בעל המשקל המקסימלי מבין כל הE_i, משקלו גדול או שווה לממוצע כל החיתוכים: $w\left(E_j\right)\geq\frac{1}{k}\sum_{i=1}^{k}w\left(E_i\right)$. מכאן נקבל עם קצת אלגברה:

$$w\left(F\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-w\left(E_j\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-\frac{1}{k}\sum_{i=1}^{k}w\left(E_i\right)=\left(1-\frac{1}{k}\right)\sum_{i=1}^{k}w\left(E_i\right)\leq\left(1-\frac{1}{k}\right)\cdot2w\left(E^*\right)$$

לכן קיבלנו סה"כ

$$w\left(F\right)\leq\left(2-\frac{2}{k}\right)w\left(E^*\right)$$

כנדרש.

■

הערה 54. החסם הדוק. כמו כן,
בשנת 2014 הציגו אלגוריתם המציג יחס קירוב של 1.2965 לבעיה.
בשנת 1994 הוכיחו כי עבור $k\geq3$ הבעיה לא ניתנת לקירוב תוך פקטור של $1+\delta$ עבור $\delta>0$, כלומר לבעיה אין $PTAS$.
בשנת 2019 הוכיחו כי תחת השערת המשחקים הייחודיים, הבעיה לא ניתנת לקירוב בפקטור שטוב מ1.20.

0>0